Introduction to Information Retrieval

Lecture 1: The IR Process, Vector Space Model & Boolean Retrieval

Avishek Anand

Information Retrieval

February 4, 2026

Course Overview

1. **Introduction to IR: Vector Space Model Boolean Retrieval** $\leftarrow$ Today
2. Indexing: Sparse and Dense
3. Ranking: BM25, Query Expansion, Language Models
4. Embeddings Transformer-based Re-rankers
5. Dense Retrieval
6. Learning to Rank

Course Overview

1. **Introduction to IR: Vector Space Model Boolean Retrieval** ← Today
2. Indexing: Sparse and Dense
3. Ranking: BM25, Query Expansion, Language Models
4. Embeddings Transformer-based Re-rankers
5. Dense Retrieval
6. Learning to Rank

Today's Goals:

- Understand what IR is and why it matters
- Learn the fundamental IR process
- Master Boolean retrieval and the vector space model
- Build your first search engine

Outline

What is Information Retrieval?

Definition

Information Retrieval (IR) is finding material (usually documents) of an unstructured nature (usually text) that satisfies an information need from within large collections (usually stored on computers).

What is Information Retrieval?

Definition

Information Retrieval (IR) is finding material (usually documents) of an unstructured nature (usually text) that satisfies an information need from within large collections (usually stored on computers).

Key Characteristics

- **Unstructured data:** Text, images, audio (vs. databases with structured records)
- **Large scale:** Billions of documents (web search, enterprise search)
- **Information need:** User wants to learn something, not exact matching
- **Relevance:** Not all results equally useful

IR vs. Database Systems

Aspect	Database Systems	IR Systems
Data	Structured (tables, schemas)	Unstructured (text, documents)
Queries	Precise (SQL)	Vague (natural language)
Matching	Exact	Approximate/Ranked
Result	All matching records	Top-k relevant documents
Failure	No results or all results	Ranked list (best effort)
Example	`SELECT * WHERE age>30`	"climate change effects"

IR vs. Database Systems

Aspect	Database Systems	IR Systems
Data	Structured (tables, schemas)	Unstructured (text, documents)
Queries	Precise (SQL)	Vague (natural language)
Matching	Exact	Approximate/Ranked
Result	All matching records	Top-k relevant documents
Failure	No results or all results	Ranked list (best effort)
Example	`SELECT * WHERE age>30`	"climate change effects"

Key Difference

Databases: *"Give me all records matching X"*
IR: *"Give me the most relevant documents about X"*

Real-World IR Applications

Web Search

- Google, Bing, DuckDuckGo
- Billions of web pages
- Sub-second response time

Real-World IR Applications

Web Search

- Google, Bing, DuckDuckGo
- Billions of web pages
- Sub-second response time

Enterprise Search

- Search company documents
- Email search
- Knowledge management

Real-World IR Applications

Web Search

- Google, Bing, DuckDuckGo
- Billions of web pages
- Sub-second response time

Enterprise Search

- Search company documents
- Email search
- Knowledge management

E-commerce

- Amazon product search
- eBay, Etsy
- Recommendation systems

Real-World IR Applications

Web Search

- Google, Bing, DuckDuckGo
- Billions of web pages
- Sub-second response time

Enterprise Search

- Search company documents
- Email search
- Knowledge management

E-commerce

- Amazon product search
- eBay, Etsy
- Recommendation systems

Specialized Domains

- Legal document search (LexisNexis)
- Academic papers (Google Scholar)
- Medical records (PubMed)
- News archives

IR is everywhere you search for information!

The Scale of Modern IR

Web Search Statistics (2024)

- **Google:** Processes 8.5+ billion searches per day
- **Index size:** 100+ trillion web pages indexed
- **Freshness:** New pages indexed within hours
- **Speed:** Results in < 0.5 seconds on average

The Scale of Modern IR

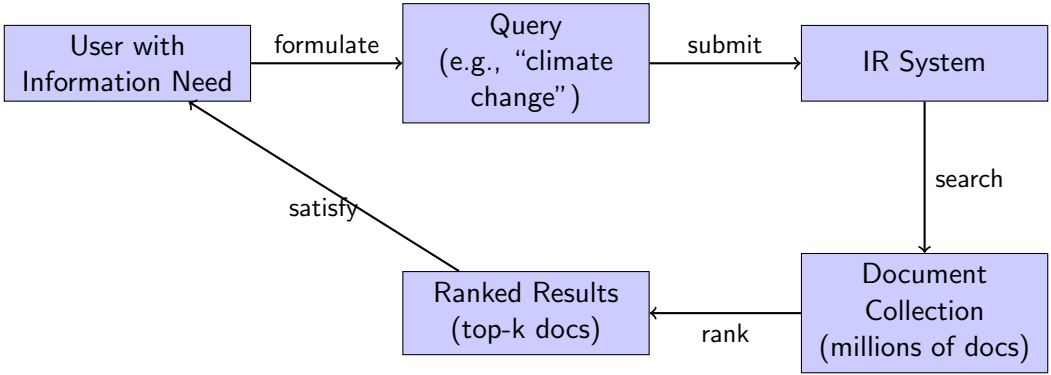
Web Search Statistics (2024)

- **Google:** Processes 8.5+ billion searches per day
- **Index size:** 100+ trillion web pages indexed
- **Freshness:** New pages indexed within hours
- **Speed:** Results in < 0.5 seconds on average

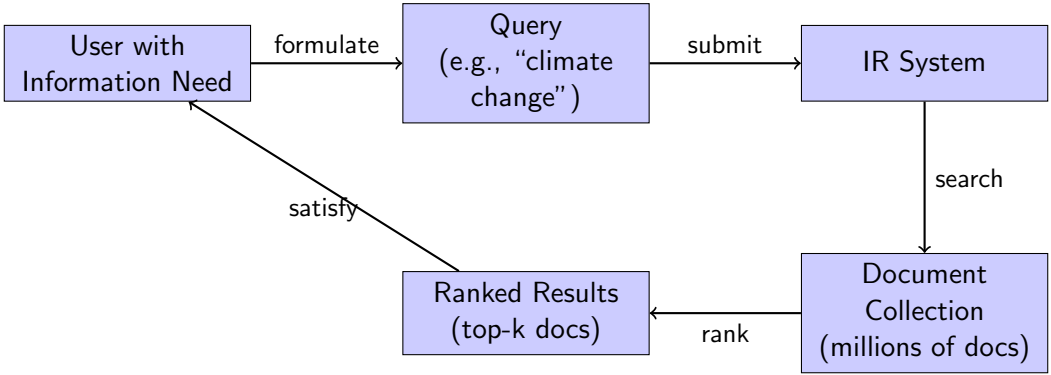
Challenges at Scale

- **Storage:** How to store 100 trillion documents?
- **Speed:** How to search in milliseconds?
- **Relevance:** How to rank billions of results?
- **Freshness:** How to keep index up-to-date?
- **Quality:** How to fight spam and misinformation?

The Information Retrieval Problem



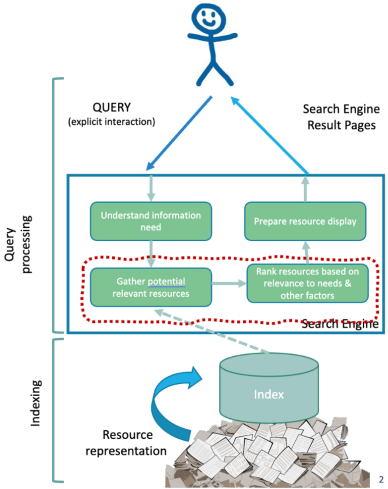
The Information Retrieval Problem



Core Questions:

1. How to represent documents and queries?
2. How to match queries to documents efficiently?

The Complete IR Pipeline



Step 1: Text Preprocessing

Goal: Convert raw text into a normalized, searchable form

Tokenization

Split text into individual tokens (words)

- Input: "The quick brown fox jumps over the lazy dog."
- Output: ["The", "quick", "brown", "fox", "jumps", "over", "the", "lazy", "dog"]

Step 1: Text Preprocessing

Goal: Convert raw text into a normalized, searchable form

Tokenization

Split text into individual tokens (words)

- Input: "The quick brown fox jumps over the lazy dog."
- Output: ["The", "quick", "brown", "fox", "jumps", "over", "the", "lazy", "dog"]

Challenges

- Punctuation: "don't" → "don" + "t" or "dont"?
- Hyphens: "state-of-the-art" → one token or multiple?
- Numbers: "$1,000.50" → keep as is or normalize?
- URLs, emails: "http://example.com" → one token?
- Different languages: Chinese/Japanese no spaces between words

Step 1: Text Preprocessing (continued)

Normalization

Convert tokens to a standard form

- **Lowercasing:** "Apple" $\rightarrow$ "apple"
- **Accent removal:** "café" $\rightarrow$ "cafe"
- **Date/number normalization:** "5/10/2024" $\rightarrow$ "2024-10-05"

Step 1: Text Preprocessing (continued)

Normalization

Convert tokens to a standard form

- **Lowercasing:** "Apple" $\rightarrow$ "apple"
- **Accent removal:** "café" $\rightarrow$ "cafe"
- **Date/number normalization:** "5/10/2024" $\rightarrow$ "2024-10-05"

Step 1: Text Preprocessing (continued)

Normalization

Convert tokens to a standard form

- **Lowercasing:** "Apple" → "apple"
- **Accent removal:** "café" → "cafe"
- **Date/number normalization:** "5/10/2024" → "2024-10-05"

Stop Word Removal

Remove common words with little meaning

- Stop words: "the", "a", "an", "is", "of", "in", ...
- Before: "the quick brown fox"
- After: "quick brown fox"
- **Caution:** Can hurt phrase queries ("to be or not to be")

Step 1: Text Preprocessing (continued)

Normalization

Convert tokens to a standard form

- **Lowercasing:** "Apple" → "apple"
- **Accent removal:** "café" → "cafe"
- **Date/number normalization:** "5/10/2024" → "2024-10-05"

Stop Word Removal

Remove common words with little meaning

- Stop words: "the", "a", "an", "is", "of", "in", ...
- Before: "the quick brown fox"
- After: "quick brown fox"
- **Caution:** Can hurt phrase queries ("to be or not to be")

Step 1: Text Preprocessing (continued)

Normalization

Convert tokens to a standard form

- **Lowercasing:** "Apple" → "apple"
- **Accent removal:** "café" → "cafe"
- **Date/number normalization:** "5/10/2024" → "2024-10-05"

Stop Word Removal

Remove common words with little meaning

- Stop words: "the", "a", "an", "is", "of", "in", ...
- Before: "the quick brown fox"
- After: "quick brown fox"
- **Caution:** Can hurt phrase queries ("to be or not to be")

Modern Trend

Many systems now *keep* stop words (storage is cheap, they can be important for some queries)

Step 1: Text Preprocessing (continued)

Normalization

Convert tokens to a standard form

- **Lowercasing:** "Apple" → "apple"
- **Accent removal:** "café" → "cafe"
- **Date/number normalization:** "5/10/2024" → "2024-10-05"

Stop Word Removal

Remove common words with little meaning

- Stop words: "the", "a", "an", "is", "of", "in", ...
- Before: "the quick brown fox"
- After: "quick brown fox"
- **Caution:** Can hurt phrase queries ("to be or not to be")

Modern Trend

Many systems now *keep* stop words (storage is cheap, they can be important for some queries)

Step 1: Stemming vs. Lemmatization

Goal: Reduce words to their base form

Stemming

Crude chopping of word endings

- **Algorithm:** Porter Stemmer
- running $\rightarrow$ run
- runs $\rightarrow$ run
- runner $\rightarrow$ runner
- university $\rightarrow$ univers

Step 1: Stemming vs. Lemmatization

Goal: Reduce words to their base form

Stemming

Crude chopping of word endings

- **Algorithm:** Porter Stemmer
- running $\rightarrow$ run
- runs $\rightarrow$ run
- runner $\rightarrow$ runner
- university $\rightarrow$ univers

Fast but imprecise

Step 1: Stemming vs. Lemmatization

Goal: Reduce words to their base form

Stemming

Crude chopping of word endings

- **Algorithm:** Porter Stemmer
- running $\rightarrow$ run
- runs $\rightarrow$ run
- runner $\rightarrow$ runner
- university $\rightarrow$ univers

Fast but imprecise

Lemmatization

Linguistic reduction to dictionary form

- **Uses:** Dictionary lookup
- running $\rightarrow$ run
- runs $\rightarrow$ run
- better $\rightarrow$ good
- am/is/are $\rightarrow$ be

Step 1: Stemming vs. Lemmatization

Goal: Reduce words to their base form

Stemming

Crude chopping of word endings

- **Algorithm:** Porter Stemmer
- running $\rightarrow$ run
- runs $\rightarrow$ run
- runner $\rightarrow$ runner
- university $\rightarrow$ univers

Fast but imprecise

Lemmatization

Linguistic reduction to dictionary form

- **Uses:** Dictionary lookup
- running $\rightarrow$ run
- runs $\rightarrow$ run
- better $\rightarrow$ good
- am/is/are $\rightarrow$ be

Accurate but slower

Step 1: Stemming vs. Lemmatization

Goal: Reduce words to their base form

Stemming

Crude chopping of word endings

- **Algorithm:** Porter Stemmer
- running → run
- runs → run
- runner → runner
- university → univers

Fast but imprecise

Lemmatization

Linguistic reduction to dictionary form

- **Uses:** Dictionary lookup
- running → run
- runs → run
- better → good
- am/is/are → be

Accurate but slower

Trade-off

- **Stemming:** Faster, good for most IR tasks
- **Lemmatization:** Better for precise matching, NLP tasks
- Modern neural methods often skip both (learn representations directly)

Step 1: Stemming vs. Lemmatization

Goal: Reduce words to their base form

Stemming

Crude chopping of word endings

- **Algorithm:** Porter Stemmer
- running → run
- runs → run
- runner → runner
- university → univers

Fast but imprecise

Lemmatization

Linguistic reduction to dictionary form

- **Uses:** Dictionary lookup
- running → run
- runs → run
- better → good
- am/is/are → be

Accurate but slower

Trade-off

- **Stemming:** Faster, good for most IR tasks
- **Lemmatization:** Better for precise matching, NLP tasks
- Modern neural methods often skip both (learn representations directly)

Step 1: Stemming vs. Lemmatization

Goal: Reduce words to their base form

Stemming

Crude chopping of word endings

- **Algorithm:** Porter Stemmer
- running → run
- runs → run
- runner → runner
- university → univers

Fast but imprecise

Lemmatization

Linguistic reduction to dictionary form

- **Uses:** Dictionary lookup
- running → run
- runs → run
- better → good
- am/is/are → be

Accurate but slower

Trade-off

- **Stemming:** Faster, good for most IR tasks
- **Lemmatization:** Better for precise matching, NLP tasks
- Modern neural methods often skip both (learn representations directly)

Preprocessing Pipeline Example

Original Text

```
The Quick Brown Fox Jumps Over The Lazy Dog!
```

Preprocessing Pipeline Example

Original Text

`The Quick Brown Fox Jumps Over The Lazy Dog!`

Tokenization:
[The, Quick, Brown, Fox, Jumps, Over, The, Lazy, Dog, !]

Lowercasing:
[the, quick, brown, fox, jumps, over, the, lazy, dog, !]

Punctuation Removal:
[the, quick, brown, fox, jumps, over, the, lazy, dog]

Stop Word Removal:
[quick, brown, fox, jumps, lazy, dog]

Stemming:
[quick, brown, fox, jump, lazi, dog]

Step 2: Building the Inverted Index

Core Data Structure: Maps each term to documents containing it

Simple Example

Documents:

- Doc1: "the cat sat on the mat"
- Doc2: "the dog sat on the log"
- Doc3: "cats and dogs are friends"

Step 2: Building the Inverted Index

Core Data Structure: Maps each term to documents containing it

Simple Example

Documents:

- Doc1: "the cat sat on the mat"
- Doc2: "the dog sat on the log"
- Doc3: "cats and dogs are friends"

Step 2: Building the Inverted Index

Core Data Structure: Maps each term to documents containing it

Simple Example

Documents:

- Doc1: "the cat sat on the mat"
- Doc2: "the dog sat on the log"
- Doc3: "cats and dogs are friends"

Term	Posting List (doc IDs)
cat	[1]
sat	[1, 2]
mat	[1]
dog	[2, 3]
log	[2]
cats	[3]
friends	[3]

Query "dog": Look up posting list → return [2, 3]

Inverted Index with Term Frequencies

Enhancement: Store not just document IDs, but also term frequencies

Term	Posting List [(docID, freq)]
cat	[(1, 1)]
sat	[(1, 1), (2, 1)]
mat	[(1, 1)]
dog	[(2, 1), (3, 1)]
the	[(1, 2), (2, 2)]

Why store frequency?

- Enables ranking (documents with term appearing more are often more relevant)
- Foundation for TF-IDF, BM25 (Lecture 3)

Inverted Index with Term Frequencies

Enhancement: Store not just document IDs, but also term frequencies

Term	Posting List [(docID, freq)]
cat	[(1, 1)]
sat	[(1, 1), (2, 1)]
mat	[(1, 1)]
dog	[(2, 1), (3, 1)]
the	[(1, 2), (2, 2)]

Why store frequency?

- Enables ranking (documents with term appearing more are often more relevant)
- Foundation for TF-IDF, BM25 (Lecture 3)

Step 3: Query Processing

Apply same preprocessing to queries as documents

Example

User Query: "Running Dogs in the Park"

1. **Tokenization:** [Running, Dogs, in, the, Park]
2. **Lowercasing:** [running, dogs, in, the, park]
3. **Stop word removal:** [running, dogs, park]
4. **Stemming:** [run, dog, park]

Step 3: Query Processing

Apply same preprocessing to queries as documents

Example

User Query: "Running Dogs in the Park"

1. **Tokenization:** [Running, Dogs, in, the, Park]
2. **Lowercasing:** [running, dogs, in, the, park]
3. **Stop word removal:** [running, dogs, park]
4. **Stemming:** [run, dog, park]

Processed Query: [run, dog, park]

Step 3: Query Processing

Apply same preprocessing to queries as documents

Example

User Query: "Running Dogs in the Park"

1. **Tokenization:** [Running, Dogs, in, the, Park]
2. **Lowercasing:** [running, dogs, in, the, park]
3. **Stop word removal:** [running, dogs, park]
4. **Stemming:** [run, dog, park]

Processed Query: [run, dog, park]

Critical Principle

Same preprocessing for documents and queries!

- Otherwise terms won't match
- If docs are stemmed, queries must be stemmed

Step 3: Query Processing

Apply same preprocessing to queries as documents

Example

User Query: "Running Dogs in the Park"

1. **Tokenization:** [Running, Dogs, in, the, Park]
2. **Lowercasing:** [running, dogs, in, the, park]
3. **Stop word removal:** [running, dogs, park]
4. **Stemming:** [run, dog, park]

Processed Query: [run, dog, park]

Critical Principle

Same preprocessing for documents and queries!

- Otherwise terms won't match
- If docs are stemmed, queries must be stemmed

Step 3: Query Processing

Apply same preprocessing to queries as documents

Example

User Query: "Running Dogs in the Park"

1. **Tokenization:** [Running, Dogs, in, the, Park]
2. **Lowercasing:** [running, dogs, in, the, park]
3. **Stop word removal:** [running, dogs, park]
4. **Stemming:** [run, dog, park]

Processed Query: [run, dog, park]

Critical Principle

Same preprocessing for documents and queries!

- Otherwise terms won't match
- If docs are stemmed, queries must be stemmed

Boolean Retrieval Model

Definition

Documents are retrieved if they satisfy a Boolean expression of query terms

Boolean Retrieval Model

Definition

Documents are retrieved if they satisfy a Boolean expression of query terms

Boolean Operators:

- **AND:** Both terms must appear (intersection)
- **OR:** Either term must appear (union)
- **NOT:** Term must not appear (negation)

Boolean Retrieval Model

Definition

Documents are retrieved if they satisfy a Boolean expression of query terms

Boolean Operators:

- **AND:** Both terms must appear (intersection)
- **OR:** Either term must appear (union)
- **NOT:** Term must not appear (negation)

Example Queries

- `climate AND change` → docs containing both "climate" and "change"
- `cat OR dog` → docs containing "cat" or "dog" or both
- `java NOT coffee` → docs with "java" but not "coffee"
- `(climate AND change) OR (global AND warming)`

Boolean Retrieval Model

Definition

Documents are retrieved if they satisfy a Boolean expression of query terms

Boolean Operators:

- **AND:** Both terms must appear (intersection)
- **OR:** Either term must appear (union)
- **NOT:** Term must not appear (negation)

Example Queries

- `climate AND change` → docs containing both "climate" and "change"
- `cat OR dog` → docs containing "cat" or "dog" or both
- `java NOT coffee` → docs with "java" but not "coffee"
- `(climate AND change) OR (global AND warming)`

Boolean Retrieval Model

Definition

Documents are retrieved if they satisfy a Boolean expression of query terms

Boolean Operators:

- **AND:** Both terms must appear (intersection)
- **OR:** Either term must appear (union)
- **NOT:** Term must not appear (negation)

Example Queries

- `climate AND change` → docs containing both "climate" and "change"
- `cat OR dog` → docs containing "cat" or "dog" or both
- `java NOT coffee` → docs with "java" but not "coffee"
- `(climate AND change) OR (global AND warming)`

Boolean Retrieval with Sets

Documents as Sets of Terms

Document Collection:

- D1: {cat, sat, mat}
- D2: {dog, sat, log}
- D3: {cat, dog, run}
- D4: {bird, fly, sky}

Boolean Retrieval with Sets

Documents as Sets of Terms

Document Collection:

- D1: {cat, sat, mat}
- D2: {dog, sat, log}
- D3: {cat, dog, run}
- D4: {bird, fly, sky}

Inverted Index:

- cat: {D1, D3}
- dog: {D2, D3}
- sat: {D1, D2}
- run: {D3}

Boolean Retrieval with Sets

Documents as Sets of Terms

Document Collection:

- D1: {cat, sat, mat}
- D2: {dog, sat, log}
- D3: {cat, dog, run}
- D4: {bird, fly, sky}

Inverted Index:

- cat: {D1, D3}
- dog: {D2, D3}
- sat: {D1, D2}
- run: {D3}

Query Processing:

- `cat AND dog`: {D1, D3} $\cap$ {D2, D3} = {D3}
- `cat OR dog`: {D1, D3} $\cup$ {D2, D3} = {D1, D2, D3}
- `cat NOT dog`: {D1, D3} - {D2, D3} = {D1}

Boolean Retrieval with Sets

Documents as Sets of Terms

Document Collection:

- D1: {cat, sat, mat}
- D2: {dog, sat, log}
- D3: {cat, dog, run}
- D4: {bird, fly, sky}

Inverted Index:

- cat: {D1, D3}
- dog: {D2, D3}
- sat: {D1, D2}
- run: {D3}

Query Processing:

- `cat AND dog`: {D1, D3} $\cap$ {D2, D3} = {D3}
- `cat OR dog`: {D1, D3} $\cup$ {D2, D3} = {D1, D2, D3}
- `cat NOT dog`: {D1, D3} - {D2, D3} = {D1}

Boolean Query Processing Algorithm

```
Input: Boolean query (e.g., "A AND B OR C")
Output: Set of matching document IDs

Parse query into expression tree
for each term in query do
    Fetch posting list from inverted index
end for
Evaluate expression bottom-up:
    - For AND: Intersect posting lists
    - For OR: Union posting lists
    - For NOT: Remove from universe
return Final set of document IDs
```

Optimization: Process smaller posting lists first

- `rare AND common`: Process "rare" first (smaller list)
- Intersection size $\leq$ min(—list1—, —list2—)

Boolean Query Processing Algorithm

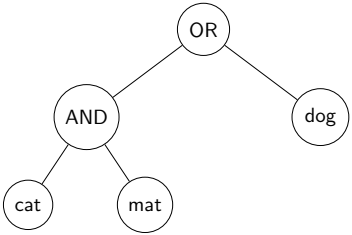
```
Input: Boolean query (e.g., "A AND B OR C")
Output: Set of matching document IDs

Parse query into expression tree
for each term in query do
    Fetch posting list from inverted index
end for
Evaluate expression bottom-up:
    - For AND: Intersect posting lists
    - For OR: Union posting lists
    - For NOT: Remove from universe
return Final set of document IDs
```

Optimization: Process smaller posting lists first

- `rare AND common`: Process "rare" first (smaller list)
- Intersection size $\leq$ min(—list1—, —list2—)

Boolean Retrieval Example

Query: `(cat AND mat) OR dog`

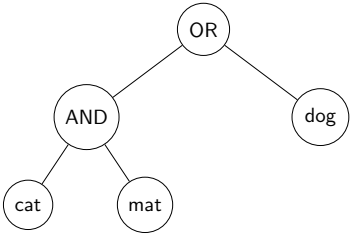


Execution:

1. `cat`: {D1, D3}
2. `mat`: {D1}
3. `cat AND mat`: {D1, D3} $\cap$ {D1} = {D1}
4. `dog`: {D2, D3}
5. `(cat AND mat) OR dog`: {D1} $\cup$ {D2, D3} = {D1, D2, D3}

Boolean Retrieval Example

Query: `(cat AND mat) OR dog`



Execution:

1. `cat`: {D1, D3}
2. `mat`: {D1}
3. `cat AND mat`: {D1, D3} $\cap$ {D1} = {D1}
4. `dog`: {D2, D3}
5. `(cat AND mat) OR dog`: {D1} $\cup$ {D2, D3} = {D1, D2, D3}

Advantages of Boolean Retrieval

✓ Advantages

- ✓ **Precise control:** Users specify exactly what they want
- ✓ **Transparent:** Clear why documents matched
- ✓ **Efficient:** Fast set operations on posting lists
- ✓ **Predictable:** Same query always returns same results
- ✓ **Expressive:** Can construct complex queries

Advantages of Boolean Retrieval

✓ Advantages

- ✓ **Precise control:** Users specify exactly what they want
- ✓ **Transparent:** Clear why documents matched
- ✓ **Efficient:** Fast set operations on posting lists
- ✓ **Predictable:** Same query always returns same results
- ✓ **Expressive:** Can construct complex queries

Advantages of Boolean Retrieval

✓ Advantages

- ✓ **Precise control:** Users specify exactly what they want
- ✓ **Transparent:** Clear why documents matched
- ✓ **Efficient:** Fast set operations on posting lists
- ✓ **Predictable:** Same query always returns same results
- ✓ **Expressive:** Can construct complex queries

Use Cases

- Legal document search (very precise requirements)
- Patent search (need exact specifications)
- Database-like queries on text
- Expert users who know what they want

Advantages of Boolean Retrieval

✓ Advantages

- ✓ **Precise control:** Users specify exactly what they want
- ✓ **Transparent:** Clear why documents matched
- ✓ **Efficient:** Fast set operations on posting lists
- ✓ **Predictable:** Same query always returns same results
- ✓ **Expressive:** Can construct complex queries

Use Cases

- Legal document search (very precise requirements)
- Patent search (need exact specifications)
- Database-like queries on text
- Expert users who know what they want

Advantages of Boolean Retrieval

✓ Advantages

- ✓ **Precise control:** Users specify exactly what they want
- ✓ **Transparent:** Clear why documents matched
- ✓ **Efficient:** Fast set operations on posting lists
- ✓ **Predictable:** Same query always returns same results
- ✓ **Expressive:** Can construct complex queries

Use Cases

- Legal document search (very precise requirements)
- Patent search (need exact specifications)
- Database-like queries on text
- Expert users who know what they want

Limitations of Boolean Retrieval

✗ Critical Limitations

✗ **No ranking**

- All matching documents equally relevant
- Can't distinguish "very relevant" from "barely relevant"

Limitations of Boolean Retrieval

✗ Critical Limitations

✗ **No ranking**

- All matching documents equally relevant
- Can't distinguish "very relevant" from "barely relevant"

✗ **All or nothing**

- `climate AND change AND effects`: No results if one term missing
- Too restrictive or too permissive

Limitations of Boolean Retrieval

✗ Critical Limitations

✗ **No ranking**

- All matching documents equally relevant
- Can't distinguish "very relevant" from "barely relevant"

✗ **All or nothing**

- `climate AND change AND effects`: No results if one term missing
- Too restrictive or too permissive

✗ **Hard to formulate queries**

- Most users don't know Boolean operators
- Hard to express fuzzy information needs

Limitations of Boolean Retrieval

✗ Critical Limitations

- ✗ **No ranking**
  - All matching documents equally relevant
  - Can't distinguish "very relevant" from "barely relevant"
- ✗ **All or nothing**
  - `climate AND change AND effects`: No results if one term missing
  - Too restrictive or too permissive
- ✗ **Hard to formulate queries**
  - Most users don't know Boolean operators
  - Hard to express fuzzy information needs
- ✗ **Result set size unpredictable**
  - Could return 0 results or 10 million
  - Users want "top 10", not "all matching"

Limitations of Boolean Retrieval

✗ Critical Limitations

- ✗ **No ranking**
  - All matching documents equally relevant
  - Can't distinguish "very relevant" from "barely relevant"
- ✗ **All or nothing**
  - `climate AND change AND effects`: No results if one term missing
  - Too restrictive or too permissive
- ✗ **Hard to formulate queries**
  - Most users don't know Boolean operators
  - Hard to express fuzzy information needs
- ✗ **Result set size unpredictable**
  - Could return 0 results or 10 million
  - Users want "top 10", not "all matching"

Limitations of Boolean Retrieval

✗ Critical Limitations

- ✗ **No ranking**
  - All matching documents equally relevant
  - Can't distinguish "very relevant" from "barely relevant"
- ✗ **All or nothing**
  - `climate AND change AND effects`: No results if one term missing
  - Too restrictive or too permissive
- ✗ **Hard to formulate queries**
  - Most users don't know Boolean operators
  - Hard to express fuzzy information needs
- ✗ **Result set size unpredictable**
  - Could return 0 results or 10 million
  - Users want "top 10", not "all matching"

Limitations of Boolean Retrieval

✗ Critical Limitations

- ✗ **No ranking**
  - All matching documents equally relevant
  - Can't distinguish "very relevant" from "barely relevant"
- ✗ **All or nothing**
  - `climate AND change AND effects`: No results if one term missing
  - Too restrictive or too permissive
- ✗ **Hard to formulate queries**
  - Most users don't know Boolean operators
  - Hard to express fuzzy information needs
- ✗ **Result set size unpredictable**
  - Could return 0 results or 10 million
  - Users want "top 10", not "all matching"

Boolean Retrieval: The Feast or Famine Problem

Too Restrictive (Famine)

- Query: `machine AND learning AND neural AND networks`
- Result: 0 documents
- Problem: One term missing kills entire result

Boolean Retrieval: The Feast or Famine Problem

Too Restrictive (Famine)

- Query: `machine AND learning AND neural AND networks`
- Result: 0 documents
- Problem: One term missing kills entire result

Solution?

- User tries: `machine AND learning`

Boolean Retrieval: The Feast or Famine Problem

Too Restrictive (Famine)

- Query: `machine AND learning AND neural AND networks`
- Result: 0 documents
- Problem: One term missing kills entire result

Solution?

- User tries: `machine AND learning`

Too Permissive (Feast)

- Query: `machine AND learning`
- Result: 1,000,000 documents
- Problem: Which 10 should I read?

Boolean Retrieval: The Feast or Famine Problem

Too Restrictive (Famine)

- Query: `machine AND learning AND neural AND networks`
- Result: 0 documents
- Problem: One term missing kills entire result

Solution?

- User tries: `machine AND learning`

Too Permissive (Feast)

- Query: `machine AND learning`
- Result: 1,000,000 documents
- Problem: Which 10 should I read?

Need:

- Ranking by relevance
- Graceful degradation
- Better matching

Boolean Retrieval: The Feast or Famine Problem

Too Restrictive (Famine)

- Query: `machine AND learning AND neural AND networks`
- Result: 0 documents
- Problem: One term missing kills entire result

Solution?

- User tries: `machine AND learning`

Too Permissive (Feast)

- Query: `machine AND learning`
- Result: 1,000,000 documents
- Problem: Which 10 should I read?

Need:

- Ranking by relevance
- Graceful degradation
- Better matching

This motivates the Vector Space Model!

From Boolean to Vector Space

Key Insight

Instead of documents being sets of terms (in/out), represent them as **vectors** with weights for each term

From Boolean to Vector Space

Key Insight

Instead of documents being sets of terms (in/out), represent them as **vectors** with weights for each term

Boolean Model:

- D1: {cat, dog}
- D2: {cat, mat}
- Binary: term present or not

From Boolean to Vector Space

Key Insight

Instead of documents being sets of terms (in/out), represent them as **vectors** with weights for each term

Boolean Model:

- D1: {cat, dog}
- D2: {cat, mat}
- Binary: term present or not

Query: `cat AND dog`

- Result: {D1}
- D2 excluded (no "dog")

From Boolean to Vector Space

Key Insight

Instead of documents being sets of terms (in/out), represent them as **vectors** with weights for each term

Boolean Model:

- D1: {cat, dog}
- D2: {cat, mat}
- Binary: term present or not

Query: `cat AND dog`

- Result: {D1}
- D2 excluded (no "dog")

Vector Space Model:

- D1: [cat: 0.5, dog: 0.5, mat: 0]
- D2: [cat: 0.7, dog: 0, mat: 0.3]
- Weighted: importance/frequency

From Boolean to Vector Space

Key Insight

Instead of documents being sets of terms (in/out), represent them as **vectors** with weights for each term

Boolean Model:

- D1: {cat, dog}
- D2: {cat, mat}
- Binary: term present or not

Query: `cat AND dog`

- Result: {D1}
- D2 excluded (no "dog")

Vector Space Model:

- D1: [cat: 0.5, dog: 0.5, mat: 0]
- D2: [cat: 0.7, dog: 0, mat: 0.3]
- Weighted: importance/frequency

Query: [cat: 0.5, dog: 0.5]

- Similarity(D1, Q) = 0.5
- Similarity(D2, Q) = 0.35
- **Ranked:** D1, then D2

From Boolean to Vector Space

Key Insight

Instead of documents being sets of terms (in/out), represent them as **vectors** with weights for each term

Boolean Model:

- D1: {cat, dog}
- D2: {cat, mat}
- Binary: term present or not

Query: `cat AND dog`

- Result: {D1}
- D2 excluded (no "dog")

Vector Space Model:

- D1: [cat: 0.5, dog: 0.5, mat: 0]
- D2: [cat: 0.7, dog: 0, mat: 0.3]
- Weighted: importance/frequency

Query: [cat: 0.5, dog: 0.5]

- Similarity(D1, Q) = 0.5
- Similarity(D2, Q) = 0.35
- **Ranked:** D1, then D2

From Boolean to Vector Space

Key Insight

Instead of documents being sets of terms (in/out), represent them as **vectors** with weights for each term

Boolean Model:

- D1: {cat, dog}
- D2: {cat, mat}
- Binary: term present or not

Query: `cat AND dog`

- Result: {D1}
- D2 excluded (no "dog")

Vector Space Model:

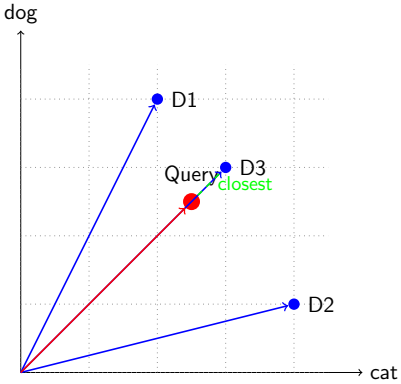
- D1: [cat: 0.5, dog: 0.5, mat: 0]
- D2: [cat: 0.7, dog: 0, mat: 0.3]
- Weighted: importance/frequency

Query: [cat: 0.5, dog: 0.5]

- Similarity(D1, Q) = 0.5
- Similarity(D2, Q) = 0.35
- **Ranked:** D1, then D2

Vector Space Model: Geometric View

Documents and queries as vectors in term space



Similarity = Angle between vectors
(Smaller angle = more similar)

Term Weighting in Vector Space Model

Question: How to assign weights to terms?

Option 1: Binary Weights

$$w_{t,d} = \begin{cases} 1 & \text{if term } t \text{ appears in doc } d \\ 0 & \text{otherwise} \end{cases}$$

- Simple but ignores frequency

Term Weighting in Vector Space Model

Question: How to assign weights to terms?

Option 1: Binary Weights

$$w_{t,d} = \begin{cases} 1 & \text{if term } t \text{ appears in doc } d \\ 0 & \text{otherwise} \end{cases}$$

- Simple but ignores frequency

Term Weighting in Vector Space Model

Question: How to assign weights to terms?

Option 1: Binary Weights

$$w_{t,d} = \begin{cases} 1 & \text{if term } t \text{ appears in doc } d \\ 0 & \text{otherwise} \end{cases}$$

- Simple but ignores frequency

Option 2: Term Frequency (TF)

$$w_{t,d} = \text{count}(t, d)$$

- Better: reflects importance
- Problem: Common words get high weight

Term Weighting in Vector Space Model

Question: How to assign weights to terms?

Option 1: Binary Weights

$$w_{t,d} = \begin{cases} 1 & \text{if term } t \text{ appears in doc } d \\ 0 & \text{otherwise} \end{cases}$$

- Simple but ignores frequency

Option 2: Term Frequency (TF)

$$w_{t,d} = \text{count}(t, d)$$

- Better: reflects importance
- Problem: Common words get high weight

Term Weighting in Vector Space Model

Question: How to assign weights to terms?

Option 1: Binary Weights

$$w_{t,d} = \begin{cases} 1 & \text{if term } t \text{ appears in doc } d \\ 0 & \text{otherwise} \end{cases}$$

- Simple but ignores frequency

Option 2: Term Frequency (TF)

$$w_{t,d} = \text{count}(t, d)$$

- Better: reflects importance
- Problem: Common words get high weight

Problem with Raw TF

Document mentioning "the" 100 times doesn't make it about "the"

Term Weighting in Vector Space Model

Question: How to assign weights to terms?

Option 1: Binary Weights

$$w_{t,d} = \begin{cases} 1 & \text{if term } t \text{ appears in doc } d \\ 0 & \text{otherwise} \end{cases}$$

- Simple but ignores frequency

Option 2: Term Frequency (TF)

$$w_{t,d} = \text{count}(t, d)$$

- Better: reflects importance
- Problem: Common words get high weight

Problem with Raw TF

Document mentioning "the" 100 times doesn't make it about "the"

TF-IDF Weighting

Idea: Balance term frequency with term rarity

TF-IDF Formula

$$w_{t,d} = \text{tf}(t, d) \times \text{idf}(t)$$

where:

- $\text{tf}(t, d) = \text{count}(t, d)$ (term frequency in document)
- $\text{idf}(t) = \log \frac{N}{\text{df}(t)}$ (inverse document frequency)
- N = total number of documents
- $\text{df}(t)$ = number of documents containing term t

TF-IDF Weighting

Idea: Balance term frequency with term rarity

TF-IDF Formula

$$w_{t,d} = \text{tf}(t, d) \times \text{idf}(t)$$

where:

- $\text{tf}(t, d) = \text{count}(t, d)$ (term frequency in document)
- $\text{idf}(t) = \log \frac{N}{\text{df}(t)}$ (inverse document frequency)
- N = total number of documents
- $\text{df}(t)$ = number of documents containing term t

TF-IDF Weighting

Idea: Balance term frequency with term rarity

TF-IDF Formula

$$w_{t,d} = \text{tf}(t, d) \times \text{idf}(t)$$

where:

- $\text{tf}(t, d) = \text{count}(t, d)$ (term frequency in document)
- $\text{idf}(t) = \log \frac{N}{\text{df}(t)}$ (inverse document frequency)
- N = total number of documents
- $\text{df}(t)$ = number of documents containing term t

Intuition:

- **High TF:** Term appears often in this document $\rightarrow$ important for this doc
- **High IDF:** Term is rare across collection $\rightarrow$ discriminative
- **High TF-IDF:** Important AND discriminative

TF-IDF Weighting

Idea: Balance term frequency with term rarity

TF-IDF Formula

$$w_{t,d} = \text{tf}(t, d) \times \text{idf}(t)$$

where:

- $\text{tf}(t, d) = \text{count}(t, d)$ (term frequency in document)
- $\text{idf}(t) = \log \frac{N}{\text{df}(t)}$ (inverse document frequency)
- N = total number of documents
- $\text{df}(t)$ = number of documents containing term t

Intuition:

- **High TF:** Term appears often in this document $\rightarrow$ important for this doc
- **High IDF:** Term is rare across collection $\rightarrow$ discriminative
- **High TF-IDF:** Important AND discriminative

TF-IDF Example

Collection: 10,000 documents

Term	tf(t,d)	df(t)	idf(t)	tf-idf
neural	5	100	$\log(10000/100) = 2.0$	10.0
network	3	500	$\log(10000/500) = 1.3$	3.9
the	50	9000	$\log(10000/9000) = 0.05$	2.5

Observations:

- "neural" appears less than "the" (5 vs. 50) but gets higher weight (10.0 vs. 2.5)
- Rare, discriminative terms weighted higher
- Common words automatically downweighted

TF-IDF Example

Collection: 10,000 documents

Term	tf(t,d)	df(t)	idf(t)	tf-idf
neural	5	100	$\log(10000/100) = 2.0$	10.0
network	3	500	$\log(10000/500) = 1.3$	3.9
the	50	9000	$\log(10000/9000) = 0.05$	2.5

Observations:

- "neural" appears less than "the" (5 vs. 50) but gets higher weight (10.0 vs. 2.5)
- Rare, discriminative terms weighted higher
- Common words automatically downweighted

Note

This is a simplified version; we'll cover refined TF-IDF and BM25 in Lecture 3

TF-IDF Example

Collection: 10,000 documents

Term	tf(t,d)	df(t)	idf(t)	tf-idf
neural	5	100	$\log(10000/100) = 2.0$	10.0
network	3	500	$\log(10000/500) = 1.3$	3.9
the	50	9000	$\log(10000/9000) = 0.05$	2.5

Observations:

- "neural" appears less than "the" (5 vs. 50) but gets higher weight (10.0 vs. 2.5)
- Rare, discriminative terms weighted higher
- Common words automatically downweighted

Note

This is a simplified version; we'll cover refined TF-IDF and BM25 in Lecture 3

Document Vectors with TF-IDF

Example Collection:

- D1: "cat cat dog"
- D2: "cat mat"
- D3: "dog bird bird"

Document Vectors with TF-IDF

Example Collection:

- D1: "cat cat dog"
- D2: "cat mat"
- D3: "dog bird bird"

Vocabulary: {cat, dog, mat, bird}

Doc	cat	dog	mat	bird
D1	$2 \times \log(3/2) = 0.35$	$1 \times \log(3/2) = 0.18$	0	0
D2	$1 \times \log(3/2) = 0.18$	0	$1 \times \log(3/1) = 0.48$	0
D3	0	$1 \times \log(3/2) = 0.18$	0	$2 \times \log(3/1) = 0.95$

Vector Representation:

- D1: [0.35, 0.18, 0, 0]
- D2: [0.18, 0, 0.48, 0]
- D3: [0, 0.18, 0, 0.95]

Document Vectors with TF-IDF

Example Collection:

- D1: "cat cat dog"
- D2: "cat mat"
- D3: "dog bird bird"

Vocabulary: {cat, dog, mat, bird}

Doc	cat	dog	mat	bird
D1	$2 \times \log(3/2) = 0.35$	$1 \times \log(3/2) = 0.18$	0	0
D2	$1 \times \log(3/2) = 0.18$	0	$1 \times \log(3/1) = 0.48$	0
D3	0	$1 \times \log(3/2) = 0.18$	0	$2 \times \log(3/1) = 0.95$

Vector Representation:

- D1: [0.35, 0.18, 0, 0]
- D2: [0.18, 0, 0.48, 0]
- D3: [0, 0.18, 0, 0.95]

Cosine Similarity

How to measure similarity between vectors?

Cosine Similarity Formula

$$\text{sim}(\vec{q}, \vec{d}) = \frac{\vec{q} \cdot \vec{d}}{|\vec{q}| \times |\vec{d}|} = \frac{\sum_t q_t \times d_t}{\sqrt{\sum_t q_t^2} \times \sqrt{\sum_t d_t^2}}$$

Cosine Similarity

How to measure similarity between vectors?

Cosine Similarity Formula

$$\text{sim}(\vec{q}, \vec{d}) = \frac{\vec{q} \cdot \vec{d}}{|\vec{q}| \times |\vec{d}|} = \frac{\sum_t q_t \times d_t}{\sqrt{\sum_t q_t^2} \times \sqrt{\sum_t d_t^2}}$$

Properties:

- Range: [-1, 1] (or [0, 1] for positive weights)
- 1 = identical vectors (same direction)
- 0 = orthogonal (no common terms)
- -1 = opposite vectors (not relevant for IR)

Cosine Similarity

How to measure similarity between vectors?

Cosine Similarity Formula

$$\text{sim}(\vec{q}, \vec{d}) = \frac{\vec{q} \cdot \vec{d}}{|\vec{q}| \times |\vec{d}|} = \frac{\sum_t q_t \times d_t}{\sqrt{\sum_t q_t^2} \times \sqrt{\sum_t d_t^2}}$$

Properties:

- Range: [-1, 1] (or [0, 1] for positive weights)
- 1 = identical vectors (same direction)
- 0 = orthogonal (no common terms)
- -1 = opposite vectors (not relevant for IR)

Why cosine instead of dot product?

- Cosine normalizes for document length
- Long documents don't automatically score higher

Cosine Similarity

How to measure similarity between vectors?

Cosine Similarity Formula

$$\text{sim}(\vec{q}, \vec{d}) = \frac{\vec{q} \cdot \vec{d}}{|\vec{q}| \times |\vec{d}|} = \frac{\sum_t q_t \times d_t}{\sqrt{\sum_t q_t^2} \times \sqrt{\sum_t d_t^2}}$$

Properties:

- Range: [-1, 1] (or [0, 1] for positive weights)
- 1 = identical vectors (same direction)
- 0 = orthogonal (no common terms)
- -1 = opposite vectors (not relevant for IR)

Why cosine instead of dot product?

- Cosine normalizes for document length
- Long documents don't automatically score higher

Cosine Similarity: Example Calculation

Query: "cat dog" $\rightarrow \vec{q} = [0.5, 0.5, 0, 0]$
Documents:

- D1: [0.35, 0.18, 0, 0]
- D2: [0.18, 0, 0.48, 0]
- D3: [0, 0.18, 0, 0.95]

Cosine Similarity: Example Calculation

Query: "cat dog" $\rightarrow \vec{q} = [0.5, 0.5, 0, 0]$
Documents:

- D1: [0.35, 0.18, 0, 0]
- D2: [0.18, 0, 0.48, 0]
- D3: [0, 0.18, 0, 0.95]

Similarity Calculations:

$$\begin{aligned}
\text{sim}(q, D1) &= \frac{(0.5 \times 0.35) + (0.5 \times 0.18)}{\sqrt{0.5^2 + 0.5^2} \times \sqrt{0.35^2 + 0.18^2}} \\
&= \frac{0.265}{0.707 \times 0.394} = 0.95 \\
\text{sim}(q, D2) &= \frac{0.5 \times 0.18}{0.707 \times 0.511} = 0.25 \\
\text{sim}(q, D3) &= \frac{0.5 \times 0.18}{0.707 \times 0.967} = 0.13
\end{aligned}$$

Ranking: D1 (0.95) ¿ D2 (0.25) ¿ D3 (0.13)

Vector Space Retrieval Algorithm

```
Input: Query q, Document collection D
Output: Ranked list of documents

Offline (Indexing):
for each document d ∈ D do
    for each term t in vocabulary do
        Compute w_{t,d} = tf(t, d) × idf(t)
    end for
    Store document vector d⃗
end for

Online (Query Processing):
for each term t in query q do
    Compute w_{t,q} = tf(t, q) × idf(t)
end for
Create query vector q⃗
for each document d do
    Compute sim(q, d) = cos(q⃗, d⃗)
end for
Sort documents by similarity (descending)
return Top-k documents
```

VSM vs. Boolean Retrieval

Aspect	Boolean	Vector Space
Matching	Exact (in/out)	Partial (weighted)
Ranking	No	Yes (by similarity)
Result size	Unpredictable	Top-k
Term weighting	Binary	TF-IDF
Query expression	Boolean operators	Term list
Robustness	Brittle	Graceful degradation
User friendliness	Expert users	General users
Relevance	All equal	Ranked by relevance

Modern Usage

- **Boolean:** Still used in legal, patent search (precision critical)
- **Vector Space:** Foundation for web search, most IR systems
- **Hybrid:** Many systems combine both approaches

VSM vs. Boolean Retrieval

Aspect	Boolean	Vector Space
Matching	Exact (in/out)	Partial (weighted)
Ranking	No	Yes (by similarity)
Result size	Unpredictable	Top-k
Term weighting	Binary	TF-IDF
Query expression	Boolean operators	Term list
Robustness	Brittle	Graceful degradation
User friendliness	Expert users	General users
Relevance	All equal	Ranked by relevance

Modern Usage

- **Boolean:** Still used in legal, patent search (precision critical)
- **Vector Space:** Foundation for web search, most IR systems
- **Hybrid:** Many systems combine both approaches

VSM vs. Boolean Retrieval

Aspect	Boolean	Vector Space
Matching	Exact (in/out)	Partial (weighted)
Ranking	No	Yes (by similarity)
Result size	Unpredictable	Top-k
Term weighting	Binary	TF-IDF
Query expression	Boolean operators	Term list
Robustness	Brittle	Graceful degradation
User friendliness	Expert users	General users
Relevance	All equal	Ranked by relevance

Modern Usage

- **Boolean:** Still used in legal, patent search (precision critical)
- **Vector Space:** Foundation for web search, most IR systems
- **Hybrid:** Many systems combine both approaches

Advantages of Vector Space Model

✓Key Advantages

1. **Ranking by relevance**
   - Documents ordered by similarity to query
   - Users get best results first

Advantages of Vector Space Model

✓Key Advantages

1. **Ranking by relevance**
   - Documents ordered by similarity to query
   - Users get best results first
2. **Partial matching**
   - Documents matching some query terms still retrieved
   - Graceful degradation (no feast/famine)

Advantages of Vector Space Model

✓Key Advantages

1. **Ranking by relevance**
   - Documents ordered by similarity to query
   - Users get best results first
2. **Partial matching**
   - Documents matching some query terms still retrieved
   - Graceful degradation (no feast/famine)
3. **Term weighting**
   - Important terms weighted higher (TF-IDF)
   - Automatic handling of common words

Advantages of Vector Space Model

✓Key Advantages

1. **Ranking by relevance**
   - Documents ordered by similarity to query
   - Users get best results first
2. **Partial matching**
   - Documents matching some query terms still retrieved
   - Graceful degradation (no feast/famine)
3. **Term weighting**
   - Important terms weighted higher (TF-IDF)
   - Automatic handling of common words
4. **Simple queries**
   - Users just type terms (no Boolean operators)
   - Natural for web search

Advantages of Vector Space Model

✓Key Advantages

1. **Ranking by relevance**
   - Documents ordered by similarity to query
   - Users get best results first
2. **Partial matching**
   - Documents matching some query terms still retrieved
   - Graceful degradation (no feast/famine)
3. **Term weighting**
   - Important terms weighted higher (TF-IDF)
   - Automatic handling of common words
4. **Simple queries**
   - Users just type terms (no Boolean operators)
   - Natural for web search
5. **Tunable**

Advantages of Vector Space Model

✓Key Advantages

1. **Ranking by relevance**
   - Documents ordered by similarity to query
   - Users get best results first
2. **Partial matching**
   - Documents matching some query terms still retrieved
   - Graceful degradation (no feast/famine)
3. **Term weighting**
   - Important terms weighted higher (TF-IDF)
   - Automatic handling of common words
4. **Simple queries**
   - Users just type terms (no Boolean operators)
   - Natural for web search
5. **Tunable**

Advantages of Vector Space Model

✓Key Advantages

1. **Ranking by relevance**
   - Documents ordered by similarity to query
   - Users get best results first
2. **Partial matching**
   - Documents matching some query terms still retrieved
   - Graceful degradation (no feast/famine)
3. **Term weighting**
   - Important terms weighted higher (TF-IDF)
   - Automatic handling of common words
4. **Simple queries**
   - Users just type terms (no Boolean operators)
   - Natural for web search
5. **Tunable**

Advantages of Vector Space Model

✓Key Advantages

1. **Ranking by relevance**
   - Documents ordered by similarity to query
   - Users get best results first
2. **Partial matching**
   - Documents matching some query terms still retrieved
   - Graceful degradation (no feast/famine)
3. **Term weighting**
   - Important terms weighted higher (TF-IDF)
   - Automatic handling of common words
4. **Simple queries**
   - Users just type terms (no Boolean operators)
   - Natural for web search
5. **Tunable**

Limitations of Vector Space Model

✗ Limitations

1. **Bag of words assumption**
   - Word order ignored: "dog bites man" = "man bites dog"
   - No phrase matching

Limitations of Vector Space Model

✗ Limitations

1. **Bag of words assumption**
   - Word order ignored: "dog bites man" = "man bites dog"
   - No phrase matching
2. **Term independence**
   - Assumes terms are independent
   - "New York" treated as "New" and "York"

Limitations of Vector Space Model

✗ Limitations

1. **Bag of words assumption**
   - Word order ignored: "dog bites man" = "man bites dog"
   - No phrase matching
2. **Term independence**
   - Assumes terms are independent
   - "New York" treated as "New" and "York"
3. **Synonymy**
   - "car" and "automobile" treated as different
   - Misses relevant documents with synonyms

Limitations of Vector Space Model

✗ Limitations

1. **Bag of words assumption**
   - Word order ignored: "dog bites man" = "man bites dog"
   - No phrase matching
2. **Term independence**
   - Assumes terms are independent
   - "New York" treated as "New" and "York"
3. **Synonymy**
   - "car" and "automobile" treated as different
   - Misses relevant documents with synonyms
4. **Polysemy**
   - "bank" (financial) vs. "bank" (river) undistinguished
   - Can retrieve irrelevant documents

Limitations of Vector Space Model

✗ Limitations

1. **Bag of words assumption**
   - Word order ignored: "dog bites man" = "man bites dog"
   - No phrase matching
2. **Term independence**
   - Assumes terms are independent
   - "New York" treated as "New" and "York"
3. **Synonymy**
   - "car" and "automobile" treated as different
   - Misses relevant documents with synonyms
4. **Polysemy**
   - "bank" (financial) vs. "bank" (river) undistinguished
   - Can retrieve irrelevant documents
5. **Scalability**
   - Computing similarity for all documents expensive
   - Need optimizations for large collections

Limitations of Vector Space Model

✗ Limitations

1. **Bag of words assumption**
   - Word order ignored: "dog bites man" = "man bites dog"
   - No phrase matching
2. **Term independence**
   - Assumes terms are independent
   - "New York" treated as "New" and "York"
3. **Synonymy**
   - "car" and "automobile" treated as different
   - Misses relevant documents with synonyms
4. **Polysemy**
   - "bank" (financial) vs. "bank" (river) undistinguished
   - Can retrieve irrelevant documents
5. **Scalability**
   - Computing similarity for all documents expensive
   - Need optimizations for large collections

Limitations of Vector Space Model

✗ Limitations

1. **Bag of words assumption**
   - Word order ignored: "dog bites man" = "man bites dog"
   - No phrase matching
2. **Term independence**
   - Assumes terms are independent
   - "New York" treated as "New" and "York"
3. **Synonymy**
   - "car" and "automobile" treated as different
   - Misses relevant documents with synonyms
4. **Polysemy**
   - "bank" (financial) vs. "bank" (river) undistinguished
   - Can retrieve irrelevant documents
5. **Scalability**
   - Computing similarity for all documents expensive
   - Need optimizations for large collections

Limitations of Vector Space Model

✗ Limitations

1. **Bag of words assumption**
   - Word order ignored: "dog bites man" = "man bites dog"
   - No phrase matching
2. **Term independence**
   - Assumes terms are independent
   - "New York" treated as "New" and "York"
3. **Synonymy**
   - "car" and "automobile" treated as different
   - Misses relevant documents with synonyms
4. **Polysemy**
   - "bank" (financial) vs. "bank" (river) undistinguished
   - Can retrieve irrelevant documents
5. **Scalability**
   - Computing similarity for all documents expensive
   - Need optimizations for large collections

From Boolean/VSM to Modern IR

Today we learned:

- Boolean retrieval: Exact matching, no ranking
- Vector space model: TF-IDF, cosine similarity, ranked results

But we still have major limitations:

- Bag of words (no semantics)
- Exact term matching only
- Simple weighting (TF-IDF)

From Boolean/VSM to Modern IR

Today we learned:

- Boolean retrieval: Exact matching, no ranking
- Vector space model: TF-IDF, cosine similarity, ranked results

But we still have major limitations:

- Bag of words (no semantics)
- Exact term matching only
- Simple weighting (TF-IDF)

Preview of Course

1. **Lecture 2 (Indexing):** How to build inverted indexes efficiently at scale
2. **Lecture 3 (Ranking):** Better weighting (BM25), query expansion, language models
3. **Lecture 4 (Transformers):** Neural re-ranking with BERT
4. **Lecture 5 (Dense Retrieval):** Semantic matching with embeddings
5. **Lecture 6 (Learning to Rank):** Learning optimal ranking from data

From Boolean/VSM to Modern IR

Today we learned:

- Boolean retrieval: Exact matching, no ranking
- Vector space model: TF-IDF, cosine similarity, ranked results

But we still have major limitations:

- Bag of words (no semantics)
- Exact term matching only
- Simple weighting (TF-IDF)

Preview of Course

1. **Lecture 2 (Indexing):** How to build inverted indexes efficiently at scale
2. **Lecture 3 (Ranking):** Better weighting (BM25), query expansion, language models
3. **Lecture 4 (Transformers):** Neural re-ranking with BERT
4. **Lecture 5 (Dense Retrieval):** Semantic matching with embeddings
5. **Lecture 6 (Learning to Rank):** Learning optimal ranking from data

From Boolean/VSM to Modern IR

Today we learned:

- Boolean retrieval: Exact matching, no ranking
- Vector space model: TF-IDF, cosine similarity, ranked results

But we still have major limitations:

- Bag of words (no semantics)
- Exact term matching only
- Simple weighting (TF-IDF)

Preview of Course

1. **Lecture 2 (Indexing):** How to build inverted indexes efficiently at scale
2. **Lecture 3 (Ranking):** Better weighting (BM25), query expansion, language models
3. **Lecture 4 (Transformers):** Neural re-ranking with BERT
4. **Lecture 5 (Dense Retrieval):** Semantic matching with embeddings
5. **Lecture 6 (Learning to Rank):** Learning optimal ranking from data

Preview: Lecture 2 — Indexing

Why Indexing Matters

- Building inverted indexes for billions of documents
- Sparse vs. dense representations
- Compression techniques
- Distributed indexing

Preview: Lecture 2 — Indexing

Why Indexing Matters

- Building inverted indexes for billions of documents
- Sparse vs. dense representations
- Compression techniques
- Distributed indexing

Preview: Lecture 2 — Indexing

Why Indexing Matters

- Building inverted indexes for billions of documents
- Sparse vs. dense representations
- Compression techniques
- Distributed indexing

Key Questions

1. How does Google index 100 trillion web pages?
2. How to keep index up-to-date with new documents?
3. How to compress posting lists for efficiency?
4. What are dense indexes and how do they differ?

Preview: Lecture 2 — Indexing

Why Indexing Matters

- Building inverted indexes for billions of documents
- Sparse vs. dense representations
- Compression techniques
- Distributed indexing

Key Questions

1. How does Google index 100 trillion web pages?
2. How to keep index up-to-date with new documents?
3. How to compress posting lists for efficiency?
4. What are dense indexes and how do they differ?

Preview: Lecture 2 — Indexing

Why Indexing Matters

- Building inverted indexes for billions of documents
- Sparse vs. dense representations
- Compression techniques
- Distributed indexing

Key Questions

1. How does Google index 100 trillion web pages?
2. How to keep index up-to-date with new documents?
3. How to compress posting lists for efficiency?
4. What are dense indexes and how do they differ?

Preview: Lecture 3 — Ranking Models

Beyond TF-IDF

- **BM25:** Probabilistic ranking with saturation
- **Language models:** Generative approach to ranking
- **Query expansion:** Automatic term expansion with RM3
- **Pseudo relevance feedback:** Learn from top results

Preview: Lecture 3 — Ranking Models

Beyond TF-IDF

- **BM25:** Probabilistic ranking with saturation
- **Language models:** Generative approach to ranking
- **Query expansion:** Automatic term expansion with RM3
- **Pseudo relevance feedback:** Learn from top results

Preview: Lecture 3 — Ranking Models

Beyond TF-IDF

- **BM25:** Probabilistic ranking with saturation
- **Language models:** Generative approach to ranking
- **Query expansion:** Automatic term expansion with RM3
- **Pseudo relevance feedback:** Learn from top results

What You'll Learn

- Why BM25 is better than TF-IDF
- How to handle term saturation and length normalization
- How to expand queries automatically
- How to implement these in PyTerrier

Preview: Lecture 3 — Ranking Models

Beyond TF-IDF

- **BM25:** Probabilistic ranking with saturation
- **Language models:** Generative approach to ranking
- **Query expansion:** Automatic term expansion with RM3
- **Pseudo relevance feedback:** Learn from top results

What You'll Learn

- Why BM25 is better than TF-IDF
- How to handle term saturation and length normalization
- How to expand queries automatically
- How to implement these in PyTerrier

Preview: Lecture 3 — Ranking Models

Beyond TF-IDF

- **BM25:** Probabilistic ranking with saturation
- **Language models:** Generative approach to ranking
- **Query expansion:** Automatic term expansion with RM3
- **Pseudo relevance feedback:** Learn from top results

What You'll Learn

- Why BM25 is better than TF-IDF
- How to handle term saturation and length normalization
- How to expand queries automatically
- How to implement these in PyTerrier

Preview: Lectures 4-6

Lecture 4: Transformer Re-rankers

- BERT and transformer architectures
- Contextualized word embeddings
- Re-ranking with neural models
- "bank" (river) vs. "bank" (financial) finally distinguished!

Preview: Lectures 4-6

Lecture 4: Transformer Re-rankers

- BERT and transformer architectures
- Contextualized word embeddings
- Re-ranking with neural models
- "bank" (river) vs. "bank" (financial) finally distinguished!

Preview: Lectures 4-6

Lecture 4: Transformer Re-rankers

- BERT and transformer architectures
- Contextualized word embeddings
- Re-ranking with neural models
- "bank" (river) vs. "bank" (financial) finally distinguished!

Lecture 5: Dense Retrieval

- Embed documents and queries in semantic space
- No more vocabulary mismatch!
- "car" and "automobile" are now similar
- Approximate nearest neighbor search

Preview: Lectures 4-6

Lecture 4: Transformer Re-rankers

- BERT and transformer architectures
- Contextualized word embeddings
- Re-ranking with neural models
- "bank" (river) vs. "bank" (financial) finally distinguished!

Lecture 5: Dense Retrieval

- Embed documents and queries in semantic space
- No more vocabulary mismatch!
- "car" and "automobile" are now similar
- Approximate nearest neighbor search

Preview: Lectures 4-6

Lecture 4: Transformer Re-rankers

- BERT and transformer architectures
- Contextualized word embeddings
- Re-ranking with neural models
- "bank" (river) vs. "bank" (financial) finally distinguished!

Lecture 5: Dense Retrieval

- Embed documents and queries in semantic space
- No more vocabulary mismatch!
- "car" and "automobile" are now similar
- Approximate nearest neighbor search

Lecture 6: Learning to Rank

Preview: Lectures 4-6

Lecture 4: Transformer Re-rankers

- BERT and transformer architectures
- Contextualized word embeddings
- Re-ranking with neural models
- "bank" (river) vs. "bank" (financial) finally distinguished!

Lecture 5: Dense Retrieval

- Embed documents and queries in semantic space
- No more vocabulary mismatch!
- "car" and "automobile" are now similar
- Approximate nearest neighbor search

Lecture 6: Learning to Rank

Preview: Lectures 4-6

Lecture 4: Transformer Re-rankers

- BERT and transformer architectures
- Contextualized word embeddings
- Re-ranking with neural models
- "bank" (river) vs. "bank" (financial) finally distinguished!

Lecture 5: Dense Retrieval

- Embed documents and queries in semantic space
- No more vocabulary mismatch!
- "car" and "automobile" are now similar
- Approximate nearest neighbor search

Lecture 6: Learning to Rank

The Modern IR Stack

Today: Boolean + Vector Space (TF-IDF, Cosine)

↓

Lecture 2: Efficient Indexing (Sparse + Dense)

↓

Lecture 3: Better Ranking (BM25, LM, Query Expansion)

↓

Lecture 4: Neural Re-ranking (BERT, Transformers)

Key Takeaways

1. Information Retrieval Fundamentals

- IR: Finding relevant unstructured documents at scale
- Different from database systems (approximate, ranked)
- Core challenges: representation, matching, ranking

Key Takeaways

1. Information Retrieval Fundamentals

- IR: Finding relevant unstructured documents at scale
- Different from database systems (approximate, ranked)
- Core challenges: representation, matching, ranking

Key Takeaways

1. Information Retrieval Fundamentals

- IR: Finding relevant unstructured documents at scale
- Different from database systems (approximate, ranked)
- Core challenges: representation, matching, ranking

2. The IR Process

- Text preprocessing: tokenization, normalization, stemming
- Inverted index: maps terms to documents
- Query processing: same preprocessing as documents

Key Takeaways

1. Information Retrieval Fundamentals

- IR: Finding relevant unstructured documents at scale
- Different from database systems (approximate, ranked)
- Core challenges: representation, matching, ranking

2. The IR Process

- Text preprocessing: tokenization, normalization, stemming
- Inverted index: maps terms to documents
- Query processing: same preprocessing as documents

Key Takeaways

1. Information Retrieval Fundamentals

- IR: Finding relevant unstructured documents at scale
- Different from database systems (approximate, ranked)
- Core challenges: representation, matching, ranking

2. The IR Process

- Text preprocessing: tokenization, normalization, stemming
- Inverted index: maps terms to documents
- Query processing: same preprocessing as documents

3. Boolean Retrieval

- Exact matching with AND/OR/NOT operators
- Fast, transparent, but no ranking

Key Takeaways

1. Information Retrieval Fundamentals

- IR: Finding relevant unstructured documents at scale
- Different from database systems (approximate, ranked)
- Core challenges: representation, matching, ranking

2. The IR Process

- Text preprocessing: tokenization, normalization, stemming
- Inverted index: maps terms to documents
- Query processing: same preprocessing as documents

3. Boolean Retrieval

- Exact matching with AND/OR/NOT operators
- Fast, transparent, but no ranking

Key Takeaways

1. Information Retrieval Fundamentals

- IR: Finding relevant unstructured documents at scale
- Different from database systems (approximate, ranked)
- Core challenges: representation, matching, ranking

2. The IR Process

- Text preprocessing: tokenization, normalization, stemming
- Inverted index: maps terms to documents
- Query processing: same preprocessing as documents

3. Boolean Retrieval

- Exact matching with AND/OR/NOT operators
- Fast, transparent, but no ranking

Key Takeaways (continued)

4. Vector Space Model

- Documents and queries as weighted term vectors
- TF-IDF weighting: term frequency $\times$ inverse document frequency
- Cosine similarity for ranking
- Solves Boolean's limitations: ranked results, partial matching

Key Takeaways (continued)

4. Vector Space Model

- Documents and queries as weighted term vectors
- TF-IDF weighting: term frequency $\times$ inverse document frequency
- Cosine similarity for ranking
- Solves Boolean's limitations: ranked results, partial matching

Key Takeaways (continued)

4. Vector Space Model

- Documents and queries as weighted term vectors
- TF-IDF weighting: term frequency $\times$ inverse document frequency
- Cosine similarity for ranking
- Solves Boolean's limitations: ranked results, partial matching

5. Evaluation

- Precision: fraction of retrieved that are relevant
- Recall: fraction of relevant that are retrieved
- MAP: average precision across queries (main metric)
- Trade-offs based on application

Key Takeaways (continued)

4. Vector Space Model

- Documents and queries as weighted term vectors
- TF-IDF weighting: term frequency $\times$ inverse document frequency
- Cosine similarity for ranking
- Solves Boolean's limitations: ranked results, partial matching

5. Evaluation

- Precision: fraction of retrieved that are relevant
- Recall: fraction of relevant that are retrieved
- MAP: average precision across queries (main metric)
- Trade-offs based on application

Key Takeaways (continued)

4. Vector Space Model

- Documents and queries as weighted term vectors
- TF-IDF weighting: term frequency $\times$ inverse document frequency
- Cosine similarity for ranking
- Solves Boolean's limitations: ranked results, partial matching

5. Evaluation

- Precision: fraction of retrieved that are relevant
- Recall: fraction of relevant that are retrieved
- MAP: average precision across queries (main metric)
- Trade-offs based on application

Remember

Key Takeaways (continued)

4. Vector Space Model

- Documents and queries as weighted term vectors
- TF-IDF weighting: term frequency $\times$ inverse document frequency
- Cosine similarity for ranking
- Solves Boolean's limitations: ranked results, partial matching

5. Evaluation

- Precision: fraction of retrieved that are relevant
- Recall: fraction of relevant that are retrieved
- MAP: average precision across queries (main metric)
- Trade-offs based on application

Remember

Assignment 1: Build Your First Search Engine

Due: Before Lecture 2

Part 1: Text Preprocessing (20 points)

- Implement tokenization, lowercasing, stop word removal
- Implement Porter stemmer OR use existing library
- Process provided document collection
- Report: vocabulary size before/after preprocessing

Assignment 1: Build Your First Search Engine

Due: Before Lecture 2

Part 1: Text Preprocessing (20 points)

- Implement tokenization, lowercasing, stop word removal
- Implement Porter stemmer OR use existing library
- Process provided document collection
- Report: vocabulary size before/after preprocessing

Assignment 1: Build Your First Search Engine

Due: Before Lecture 2

Part 1: Text Preprocessing (20 points)

- Implement tokenization, lowercasing, stop word removal
- Implement Porter stemmer OR use existing library
- Process provided document collection
- Report: vocabulary size before/after preprocessing

Part 2: Boolean Retrieval (30 points)

- Build inverted index from preprocessed documents
- Implement AND, OR, NOT query processing
- Test on 10 queries, show results
- Analyze: when does Boolean work well/poorly?

Assignment 1: Build Your First Search Engine

Due: Before Lecture 2

Part 1: Text Preprocessing (20 points)

- Implement tokenization, lowercasing, stop word removal
- Implement Porter stemmer OR use existing library
- Process provided document collection
- Report: vocabulary size before/after preprocessing

Part 2: Boolean Retrieval (30 points)

- Build inverted index from preprocessed documents
- Implement AND, OR, NOT query processing
- Test on 10 queries, show results
- Analyze: when does Boolean work well/poorly?

Assignment 1: Build Your First Search Engine

Due: Before Lecture 2

Part 1: Text Preprocessing (20 points)

- Implement tokenization, lowercasing, stop word removal
- Implement Porter stemmer OR use existing library
- Process provided document collection
- Report: vocabulary size before/after preprocessing

Part 2: Boolean Retrieval (30 points)

- Build inverted index from preprocessed documents
- Implement AND, OR, NOT query processing
- Test on 10 queries, show results
- Analyze: when does Boolean work well/poorly?

Assignment 1 (continued)

Part 3: Vector Space Model (40 points)

- Implement TF-IDF weighting
- Implement cosine similarity
- Build ranked retrieval system
- Compare with Boolean on same 10 queries
- Report precision@10, recall for both systems

Assignment 1 (continued)

Part 3: Vector Space Model (40 points)

- Implement TF-IDF weighting
- Implement cosine similarity
- Build ranked retrieval system
- Compare with Boolean on same 10 queries
- Report precision@10, recall for both systems

Assignment 1 (continued)

Part 3: Vector Space Model (40 points)

- Implement TF-IDF weighting
- Implement cosine similarity
- Build ranked retrieval system
- Compare with Boolean on same 10 queries
- Report precision@10, recall for both systems

Part 4: Analysis (10 points)

- Show examples where VSM outperforms Boolean
- Show examples where Boolean might be preferred
- Discuss trade-offs: precision vs. recall
- Identify limitations of your implementation

Assignment 1 (continued)

Part 3: Vector Space Model (40 points)

- Implement TF-IDF weighting
- Implement cosine similarity
- Build ranked retrieval system
- Compare with Boolean on same 10 queries
- Report precision@10, recall for both systems

Part 4: Analysis (10 points)

- Show examples where VSM outperforms Boolean
- Show examples where Boolean might be preferred
- Discuss trade-offs: precision vs. recall
- Identify limitations of your implementation

Assignment 1 (continued)

Part 3: Vector Space Model (40 points)

- Implement TF-IDF weighting
- Implement cosine similarity
- Build ranked retrieval system
- Compare with Boolean on same 10 queries
- Report precision@10, recall for both systems

Part 4: Analysis (10 points)

- Show examples where VSM outperforms Boolean
- Show examples where Boolean might be preferred
- Discuss trade-offs: precision vs. recall
- Identify limitations of your implementation

Starter Code and Dataset

Provided Materials

- **Dataset:** TREC Disks 1-2 subset (10,000 documents) OR Cranfield collection
- **Queries:** 50 test queries with relevance judgments
- **Starter code:** Python skeleton with TODOs
- **Evaluation script:** Compute precision, recall, MAP automatically

Starter Code and Dataset

Provided Materials

- **Dataset:** TREC Disks 1-2 subset (10,000 documents) OR Cranfield collection
- **Queries:** 50 test queries with relevance judgments
- **Starter code:** Python skeleton with TODOs
- **Evaluation script:** Compute precision, recall, MAP automatically

Starter Code and Dataset

Provided Materials

- **Dataset:** TREC Disks 1-2 subset (10,000 documents) OR Cranfield collection
- **Queries:** 50 test queries with relevance judgments
- **Starter code:** Python skeleton with TODOs
- **Evaluation script:** Compute precision, recall, MAP automatically

Tips

- Start with small subset (100 docs) for debugging
- Use Python dictionaries for inverted index
- Test each component independently before integration
- Compare your TF-IDF values with examples in lecture
- Don't reinvent the wheel: use NLTK for tokenization, stemming

Starter Code and Dataset

Provided Materials

- **Dataset:** TREC Disks 1-2 subset (10,000 documents) OR Cranfield collection
- **Queries:** 50 test queries with relevance judgments
- **Starter code:** Python skeleton with TODOs
- **Evaluation script:** Compute precision, recall, MAP automatically

Tips

- Start with small subset (100 docs) for debugging
- Use Python dictionaries for inverted index
- Test each component independently before integration
- Compare your TF-IDF values with examples in lecture
- Don't reinvent the wheel: use NLTK for tokenization, stemming

Starter Code and Dataset

Provided Materials

- **Dataset:** TREC Disks 1-2 subset (10,000 documents) OR Cranfield collection
- **Queries:** 50 test queries with relevance judgments
- **Starter code:** Python skeleton with TODOs
- **Evaluation script:** Compute precision, recall, MAP automatically

Tips

- Start with small subset (100 docs) for debugging
- Use Python dictionaries for inverted index
- Test each component independently before integration
- Compare your TF-IDF values with examples in lecture
- Don't reinvent the wheel: use NLTK for tokenization, stemming

Next Class: Indexing

Preparation

Read:

- Manning et al., Introduction to Information Retrieval, Chapter 4 (Index Construction)
- Zobel Moffat (2006): "Inverted files for text search engines"

Next Class: Indexing

Preparation

Read:

- Manning et al., Introduction to Information Retrieval, Chapter 4 (Index Construction)
- Zobel Moffat (2006): "Inverted files for text search engines"

Skim:

- Original inverted index paper (historical perspective)

Next Class: Indexing

Preparation

Read:

- Manning et al., Introduction to Information Retrieval, Chapter 4 (Index Construction)
- Zobel Moffat (2006): "Inverted files for text search engines"

Skim:

- Original inverted index paper (historical perspective)

Next Class: Indexing

Preparation

Read:

- Manning et al., Introduction to Information Retrieval, Chapter 4 (Index Construction)
- Zobel Moffat (2006): "Inverted files for text search engines"

Skim:

- Original inverted index paper (historical perspective)

What We'll Cover

- How to build indexes for billions of documents
- Block-based indexing algorithms
- Index compression techniques

Next Class: Indexing

Preparation

Read:

- Manning et al., Introduction to Information Retrieval, Chapter 4 (Index Construction)
- Zobel Moffat (2006): "Inverted files for text search engines"

Skim:

- Original inverted index paper (historical perspective)

What We'll Cover

- How to build indexes for billions of documents
- Block-based indexing algorithms
- Index compression techniques

Next Class: Indexing

Preparation

Read:

- Manning et al., Introduction to Information Retrieval, Chapter 4 (Index Construction)
- Zobel Moffat (2006): "Inverted files for text search engines"

Skim:

- Original inverted index paper (historical perspective)

What We'll Cover

- How to build indexes for billions of documents
- Block-based indexing algorithms
- Index compression techniques

Questions?

Questions?

Office Hours: [Your time]

Email: [Your email]

Course Website: [Your URL]

Start early on the assignment!

Backup: TF-IDF Variants

Log-Frequency TF

$$\text{tf}(t, d) = \begin{cases} 1 + \log(\text{count}(t, d)) & \text{if count} > 0 \\ 0 & \text{otherwise} \end{cases}$$

- Dampens effect of high term frequencies
- 10 occurrences not 10× more important than 1

Backup: TF-IDF Variants

Log-Frequency TF

$$\text{tf}(t, d) = \begin{cases} 1 + \log(\text{count}(t, d)) & \text{if count} > 0 \\ 0 & \text{otherwise} \end{cases}$$

- Dampens effect of high term frequencies
- 10 occurrences not 10× more important than 1

Backup: TF-IDF Variants

Log-Frequency TF

$$\text{tf}(t, d) = \begin{cases} 1 + \log(\text{count}(t, d)) & \text{if count} > 0 \\ 0 & \text{otherwise} \end{cases}$$

- Dampens effect of high term frequencies
- 10 occurrences not $10\times$ more important than 1

Augmented TF (Length Normalization)

$$\text{tf}(t, d) = 0.5 + 0.5 \times \frac{\text{count}(t, d)}{\max_{t'} \text{count}(t', d)}$$

- Normalizes by maximum term frequency in document
- Prevents bias toward long documents

Backup: TF-IDF Variants

Log-Frequency TF

$$\text{tf}(t, d) = \begin{cases} 1 + \log(\text{count}(t, d)) & \text{if count} > 0 \\ 0 & \text{otherwise} \end{cases}$$

- Dampens effect of high term frequencies
- 10 occurrences not $10\times$ more important than 1

Augmented TF (Length Normalization)

$$\text{tf}(t, d) = 0.5 + 0.5 \times \frac{\text{count}(t, d)}{\max_{t'} \text{count}(t', d)}$$

- Normalizes by maximum term frequency in document
- Prevents bias toward long documents

Backup: TF-IDF Variants

Log-Frequency TF

$$\text{tf}(t, d) = \begin{cases} 1 + \log(\text{count}(t, d)) & \text{if count} > 0 \\ 0 & \text{otherwise} \end{cases}$$

- Dampens effect of high term frequencies
- 10 occurrences not 10× more important than 1

Augmented TF (Length Normalization)

$$\text{tf}(t, d) = 0.5 + 0.5 \times \frac{\text{count}(t, d)}{\max_{t'} \text{count}(t', d)}$$

- Normalizes by maximum term frequency in document
- Prevents bias toward long documents

Probabilistic IDF

$$\text{idf}(t) = \log \frac{N - \text{df}(t) + 0.5}{\text{df}(t) + 0.5}$$

- Used in BM25 (Lecture 3)
- Better theoretical foundation

Backup: TF-IDF Variants

Log-Frequency TF

$$\text{tf}(t, d) = \begin{cases} 1 + \log(\text{count}(t, d)) & \text{if count} > 0 \\ 0 & \text{otherwise} \end{cases}$$

- Dampens effect of high term frequencies
- 10 occurrences not 10× more important than 1

Augmented TF (Length Normalization)

$$\text{tf}(t, d) = 0.5 + 0.5 \times \frac{\text{count}(t, d)}{\max_{t'} \text{count}(t', d)}$$

- Normalizes by maximum term frequency in document
- Prevents bias toward long documents

Probabilistic IDF

$$\text{idf}(t) = \log \frac{N - \text{df}(t) + 0.5}{\text{df}(t) + 0.5}$$

- Used in BM25 (Lecture 3)
- Better theoretical foundation

Backup: TF-IDF Variants

Log-Frequency TF

$$\text{tf}(t, d) = \begin{cases} 1 + \log(\text{count}(t, d)) & \text{if count} > 0 \\ 0 & \text{otherwise} \end{cases}$$

- Dampens effect of high term frequencies
- 10 occurrences not 10× more important than 1

Augmented TF (Length Normalization)

$$\text{tf}(t, d) = 0.5 + 0.5 \times \frac{\text{count}(t, d)}{\max_{t'} \text{count}(t', d)}$$

- Normalizes by maximum term frequency in document
- Prevents bias toward long documents

Probabilistic IDF

$$\text{idf}(t) = \log \frac{N - \text{df}(t) + 0.5}{\text{df}(t) + 0.5}$$

- Used in BM25 (Lecture 3)
- Better theoretical foundation

Backup: Advanced Preprocessing

Named Entity Recognition (NER)

Identify and preserve named entities

- "Barack Obama" $\rightarrow$ single entity
- "New York City" $\rightarrow$ single entity
- Important for proper nouns

Backup: Advanced Preprocessing

Named Entity Recognition (NER)

Identify and preserve named entities

- "Barack Obama" $\rightarrow$ single entity
- "New York City" $\rightarrow$ single entity
- Important for proper nouns

Backup: Advanced Preprocessing

Named Entity Recognition (NER)

Identify and preserve named entities

- "Barack Obama" $\rightarrow$ single entity
- "New York City" $\rightarrow$ single entity
- Important for proper nouns

Compound Word Splitting

Handle compound words (especially in German, Dutch)

- "Informationssystem" $\rightarrow$ "Information" + "System"
- Language-specific rules

Backup: Advanced Preprocessing

Named Entity Recognition (NER)

Identify and preserve named entities

- "Barack Obama" $\rightarrow$ single entity
- "New York City" $\rightarrow$ single entity
- Important for proper nouns

Compound Word Splitting

Handle compound words (especially in German, Dutch)

- "Informationssystem" $\rightarrow$ "Information" + "System"
- Language-specific rules

Backup: Advanced Preprocessing

Named Entity Recognition (NER)

Identify and preserve named entities

- "Barack Obama" $\rightarrow$ single entity
- "New York City" $\rightarrow$ single entity
- Important for proper nouns

Compound Word Splitting

Handle compound words (especially in German, Dutch)

- "Informationssystem" $\rightarrow$ "Information" + "System"
- Language-specific rules

Case Folding Exceptions

Preserve case for certain terms

- Acronyms: "USA", "IBM"
- Names: "Bush" (president) vs. "bush" (plant)
- Context-dependent decision

Backup: Advanced Preprocessing

Named Entity Recognition (NER)

Identify and preserve named entities

- "Barack Obama" $\rightarrow$ single entity
- "New York City" $\rightarrow$ single entity
- Important for proper nouns

Compound Word Splitting

Handle compound words (especially in German, Dutch)

- "Informationssystem" $\rightarrow$ "Information" + "System"
- Language-specific rules

Case Folding Exceptions

Preserve case for certain terms

- Acronyms: "USA", "IBM"
- Names: "Bush" (president) vs. "bush" (plant)
- Context-dependent decision

Backup: Advanced Preprocessing

Named Entity Recognition (NER)

Identify and preserve named entities

- "Barack Obama" $\rightarrow$ single entity
- "New York City" $\rightarrow$ single entity
- Important for proper nouns

Compound Word Splitting

Handle compound words (especially in German, Dutch)

- "Informationssystem" $\rightarrow$ "Information" + "System"
- Language-specific rules

Case Folding Exceptions

Preserve case for certain terms

- Acronyms: "USA", "IBM"
- Names: "Bush" (president) vs. "bush" (plant)
- Context-dependent decision

Backup: Alternative Similarity Measures

Euclidean Distance

$$\text{dist}(\vec{q}, \vec{d}) = \sqrt{\sum_t (q_t - d_t)^2}$$

- ✗ Sensitive to document length
- ✗ Not normalized
- Rarely used in IR

Backup: Alternative Similarity Measures

Euclidean Distance

$$\text{dist}(\vec{q}, \vec{d}) = \sqrt{\sum_t (q_t - d_t)^2}$$

- ✗ Sensitive to document length
- ✗ Not normalized
- Rarely used in IR

Backup: Alternative Similarity Measures

Euclidean Distance

$$\text{dist}(\vec{q}, \vec{d}) = \sqrt{\sum_t (q_t - d_t)^2}$$

- ✗ Sensitive to document length
- ✗ Not normalized
- Rarely used in IR

Jaccard Similarity

$$\text{Jaccard}(q, d) = \frac{|q \cap d|}{|q \cup d|}$$

- Set-based similarity
- ✗ Ignores term frequencies
- Used for duplicate detection

Backup: Alternative Similarity Measures

Euclidean Distance

$$\text{dist}(\vec{q}, \vec{d}) = \sqrt{\sum_t (q_t - d_t)^2}$$

- ✗ Sensitive to document length
- ✗ Not normalized
- Rarely used in IR

Jaccard Similarity

$$\text{Jaccard}(q, d) = \frac{|q \cap d|}{|q \cup d|}$$

- Set-based similarity
- ✗ Ignores term frequencies
- Used for duplicate detection

Backup: Alternative Similarity Measures

Euclidean Distance

$$\text{dist}(\vec{q}, \vec{d}) = \sqrt{\sum_t (q_t - d_t)^2}$$

- ✗ Sensitive to document length
- ✗ Not normalized
- Rarely used in IR

Jaccard Similarity

$$\text{Jaccard}(q, d) = \frac{|q \cap d|}{|q \cup d|}$$

- Set-based similarity
- ✗ Ignores term frequencies
- Used for duplicate detection

Dice Coefficient

$$\text{Dice}(q, d) = \frac{2|q \cap d|}{|q| + |d|}$$

- Similar to Jaccard
- Weighs shared terms more heavily

Backup: Alternative Similarity Measures

Euclidean Distance

$$\text{dist}(\vec{q}, \vec{d}) = \sqrt{\sum_t (q_t - d_t)^2}$$

- ✗ Sensitive to document length
- ✗ Not normalized
- Rarely used in IR

Jaccard Similarity

$$\text{Jaccard}(q, d) = \frac{|q \cap d|}{|q \cup d|}$$

- Set-based similarity
- ✗ Ignores term frequencies
- Used for duplicate detection

Dice Coefficient

$$\text{Dice}(q, d) = \frac{2|q \cap d|}{|q| + |d|}$$

- Similar to Jaccard
- Weighs shared terms more heavily

Backup: Alternative Similarity Measures

Euclidean Distance

$$\text{dist}(\vec{q}, \vec{d}) = \sqrt{\sum_t (q_t - d_t)^2}$$

- ✗ Sensitive to document length
- ✗ Not normalized
- Rarely used in IR

Jaccard Similarity

$$\text{Jaccard}(q, d) = \frac{|q \cap d|}{|q \cup d|}$$

- Set-based similarity
- ✗ Ignores term frequencies
- Used for duplicate detection

Dice Coefficient

$$\text{Dice}(q, d) = \frac{2|q \cap d|}{|q| + |d|}$$

- Similar to Jaccard
- Weighs shared terms more heavily

Backup: Test Collections

Standard IR Test Collections

- **TREC:** Text REtrieval Conference collections (various domains)
- **Cranfield:** Classic small collection (1400 docs, aerodynamics)
- **CACM:** Computer science abstracts (3200 docs)
- **Reuters:** News articles (800,000+ docs)
- **ClueWeb:** Web crawl (1+ billion pages)
- **MS MARCO:** Microsoft passage ranking (8.8M passages)

Backup: Test Collections

Standard IR Test Collections

- **TREC:** Text REtrieval Conference collections (various domains)
- **Cranfield:** Classic small collection (1400 docs, aerodynamics)
- **CACM:** Computer science abstracts (3200 docs)
- **Reuters:** News articles (800,000+ docs)
- **ClueWeb:** Web crawl (1+ billion pages)
- **MS MARCO:** Microsoft passage ranking (8.8M passages)

Backup: Test Collections

Standard IR Test Collections

- **TREC:** Text REtrieval Conference collections (various domains)
- **Cranfield:** Classic small collection (1400 docs, aerodynamics)
- **CACM:** Computer science abstracts (3200 docs)
- **Reuters:** News articles (800,000+ docs)
- **ClueWeb:** Web crawl (1+ billion pages)
- **MS MARCO:** Microsoft passage ranking (8.8M passages)

Components of Test Collection

1. **Documents:** The corpus to search
2. **Topics/Queries:** Information needs
3. **Relevance judgments (qrels):** Which docs are relevant to which queries
4. Usually created by human assessors (expensive!)

Backup: Test Collections

Standard IR Test Collections

- **TREC:** Text REtrieval Conference collections (various domains)
- **Cranfield:** Classic small collection (1400 docs, aerodynamics)
- **CACM:** Computer science abstracts (3200 docs)
- **Reuters:** News articles (800,000+ docs)
- **ClueWeb:** Web crawl (1+ billion pages)
- **MS MARCO:** Microsoft passage ranking (8.8M passages)

Components of Test Collection

1. **Documents:** The corpus to search
2. **Topics/Queries:** Information needs
3. **Relevance judgments (qrels):** Which docs are relevant to which queries
4. Usually created by human assessors (expensive!)

Backup: Test Collections

Standard IR Test Collections

- **TREC:** Text REtrieval Conference collections (various domains)
- **Cranfield:** Classic small collection (1400 docs, aerodynamics)
- **CACM:** Computer science abstracts (3200 docs)
- **Reuters:** News articles (800,000+ docs)
- **ClueWeb:** Web crawl (1+ billion pages)
- **MS MARCO:** Microsoft passage ranking (8.8M passages)

Components of Test Collection

1. **Documents:** The corpus to search
2. **Topics/Queries:** Information needs
3. **Relevance judgments (qrels):** Which docs are relevant to which queries
4. Usually created by human assessors (expensive!)